NAME: Atchaya Vharsne S

REG. NO.: 192524185

COURSE CODE: CSA0401

COURSE NAME: OPERATING SYSTEMS

CO5 AT2 Pseudocode Writing Task (Algorithm Design)

1. File System Space Management Algorithm

```
BEGIN
   SET threshold = 80
   usage = GET_DISK_USAGE()
   IF usage > threshold THEN
      IDENTIFY all log_files
      SORT log_files BY size DESCENDING
      FOR each file IN log_files DO
         IF file is important THEN
            COMPRESS or ROTATE file
         ELSE
            DELETE file
         ENDIF
         usage = GET_DISK_USAGE()
         IF usage <= threshold THEN
            BREAK
```

ENDIF

ENDFOR

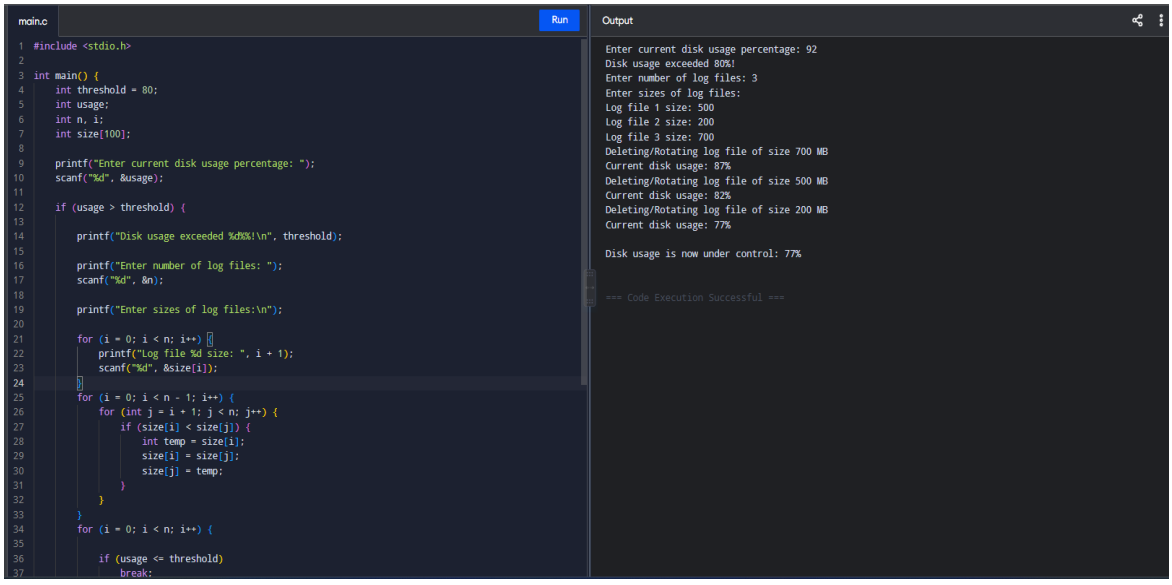
ENDIF

END

Working

1. The algorithm first checks the current disk utilization.
2. If disk usage is 80% or less, no action is taken.
3. If usage exceeds 80%, all log files are identified.
4. The files are sorted in descending order of size, so larger files are handled first.
5. Old or unnecessary logs are deleted, while important logs can be compressed or rotated.
6. After each operation, disk usage is checked again.
7. The process stops once disk utilization falls to 80% or below.

Result: This algorithm automatically prevents disk overflow by monitoring disk space and managing log files efficiently.



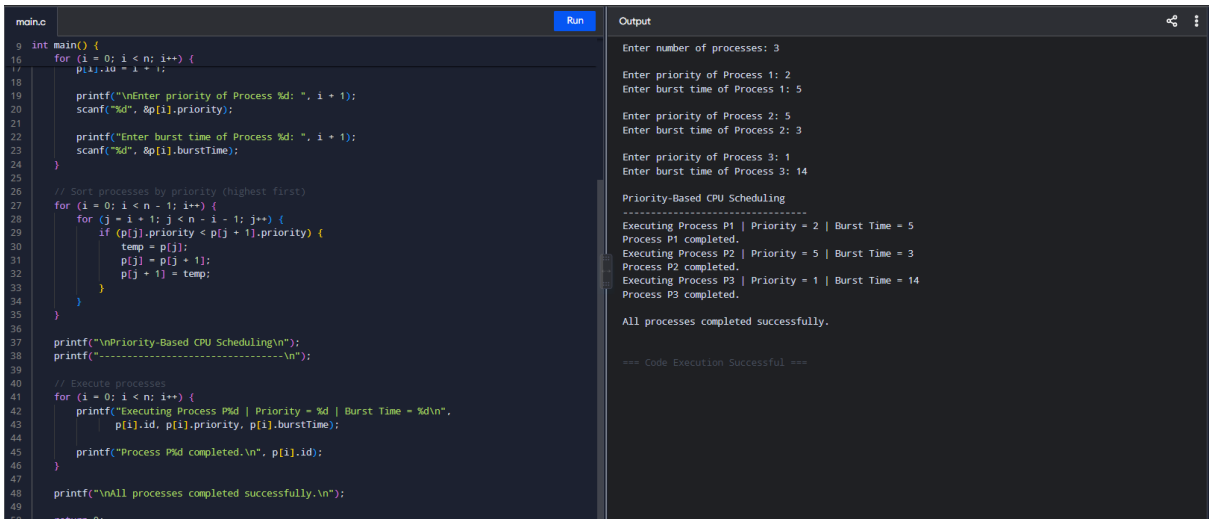
2. **Priority-Based CPU Scheduling Algorithm**

```
BEGIN
INPUT number_of_processes
FOR each process DO
    INPUT Process_ID
    INPUT Priority
    INPUT Burst_Time
    SET Remaining_Time = Burst_Time
ENDFOR
WHILE there are incomplete processes DO
    SORT ready_queue by Priority in descending order
    SELECT process with highest Priority
    ALLOCATE CPU for one time quantum
    EXECUTE selected process
    DECREASE Remaining_Time
    IF Remaining_Time = 0 THEN
        MARK process as Completed
        REMOVE process from ready_queue
    ELSE
        REDUCE Priority slightly
        ADD process back to ready_queue
    ENDIF
ENDWHILE
DISPLAY "All processes completed"
```

END

Working

1. The algorithm accepts a list of processes with their priority levels.
2. The processes are sorted in descending order of priority.
3. The process with the highest priority is selected first.
4. CPU time is allocated to the selected process.
5. The process is executed.
6. If the process is completed, it is removed from the process list.
7. If it is not completed, its priority is slightly reduced and it is added back to the process list.
8. The process list is sorted again based on priority.
9. The algorithm continues until all processes are completed.



Result: The CPU gives preference to high-priority processes while slightly reducing the priority of incomplete processes to avoid starvation and improve overall CPU efficiency.

3. **RAM Allocation for Multiple Virtual Machines**

```
BEGIN
   SET total_RAM = 16 GB
   SET host_reserved = 4 GB
   SET available_RAM = total_RAM - host_reserved
   INPUT number_of_VMs
   SET min_RAM = 2 GB
   FOR i = 1 TO number_of_VMs DO
      IF available_RAM >= min_RAM THEN
         ALLOCATE min_RAM to VM[i]
         available_RAM = available_RAM - min_RAM
         PRINT "RAM allocated to VM", i
      ELSE
         PRINT "Insufficient memory for VM", i
      ENDIF
   ENDFOR
   PRINT "Remaining available RAM =", available_RAM
END
```

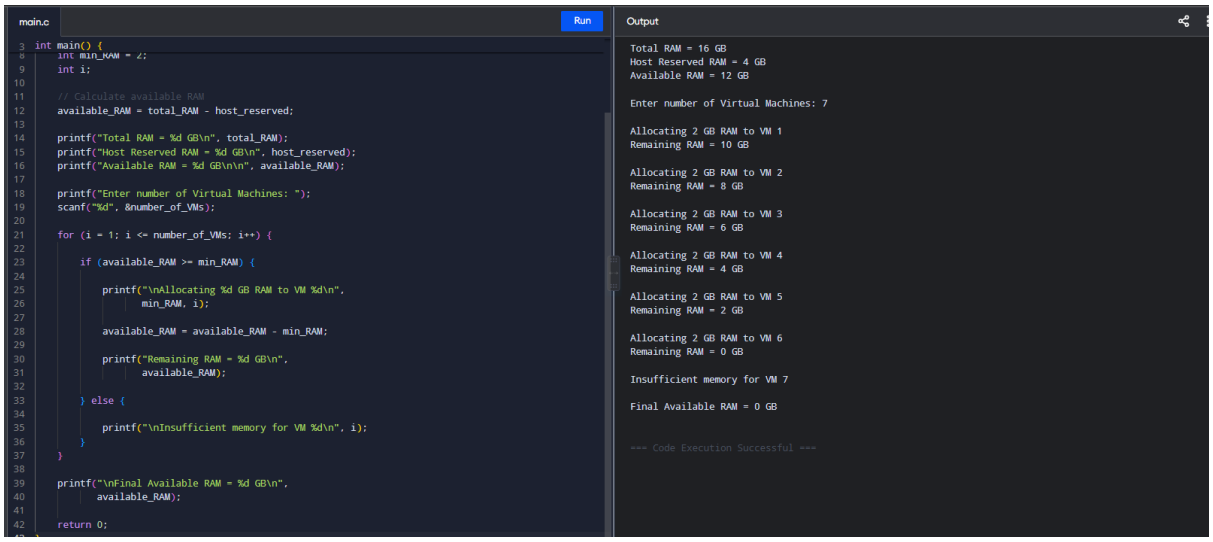
Working

1. The system has a total RAM of 16 GB.
2. 4 GB is reserved for the host operating system.
3. Therefore, the available RAM for virtual machines is:

Available RAM = 16 GB - 4 GB = 12 GB

4. The number of virtual machines is taken as input.
5. Each VM requires a minimum of 2 GB RAM.
6. Before allocating RAM, the algorithm checks whether enough memory is available.
7. If at least 2 GB is available, RAM is allocated to the VM and the available RAM is reduced.
8. If sufficient memory is not available, the system displays "Insufficient memory".
9. Finally, the remaining RAM is displayed.

Result: This algorithm efficiently distributes available RAM among multiple virtual machines while ensuring that the host system always retains its reserved memory.



4. **Hypervisor Selection Algorithm**

```
BEGIN
   INPUT performance_required
   INPUT budget
   INPUT hardware_access
   IF performance_required = HIGH AND hardware_access = TRUE THEN
      SELECT Type_1_Hyperviso
   ELSE IF budget = LOW THEN
      SELECT Type_2_Hypervisor
   ELSE
      SELECT Type_1_Hypervisor
   ENDIF
   PRINT selected_hypervisor
END
```

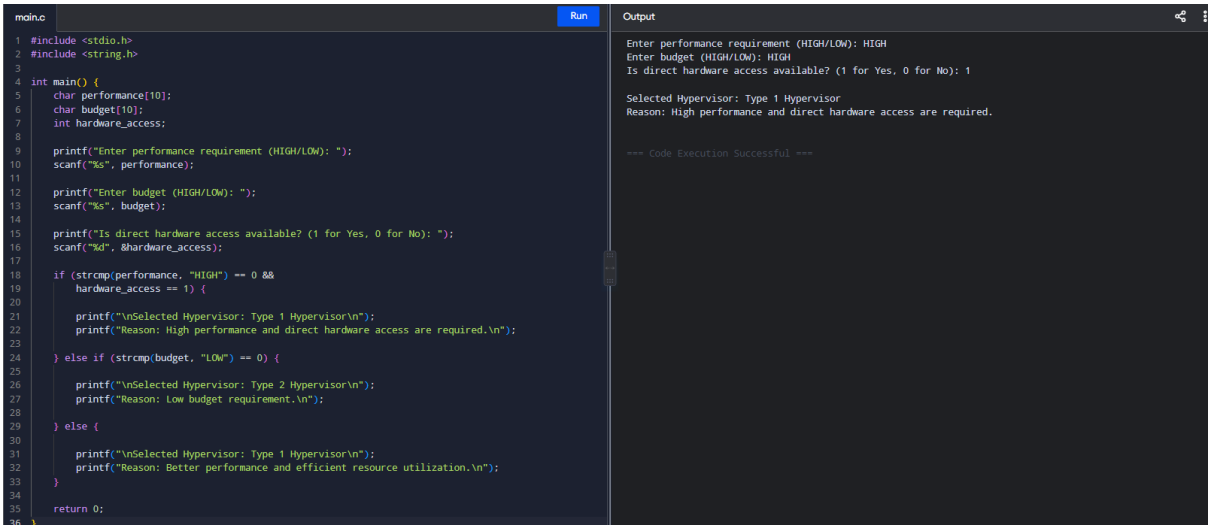
Working

1. The algorithm takes three inputs:
2. Performance requirement
3. Available budget
4. Hardware access requirement
5. If high performance is required and direct hardware access is available, the algorithm selects a Type 1 Hypervisor.
6. If the budget is low, it selects a Type 2 Hypervisor, which is generally suitable for smaller-scale or less demanding environments.
7. In all other cases, the algorithm selects a Type 1 Hypervisor for better resource utilization and performance.

Result

Type 1 Hypervisor: Selected for high-performance environments and efficient resource utilization.

Type 2 Hypervisor: Selected when cost or budget constraints are the main concern.



5. Resource Limiting Using CPU and Memory Controls

BEGIN

INPUT application_list

FOR each app IN application_list DO

CREATE cgroup for app

SET CPU_limit for app

SET memory_limit for app

ASSIGN app process to its cgroup

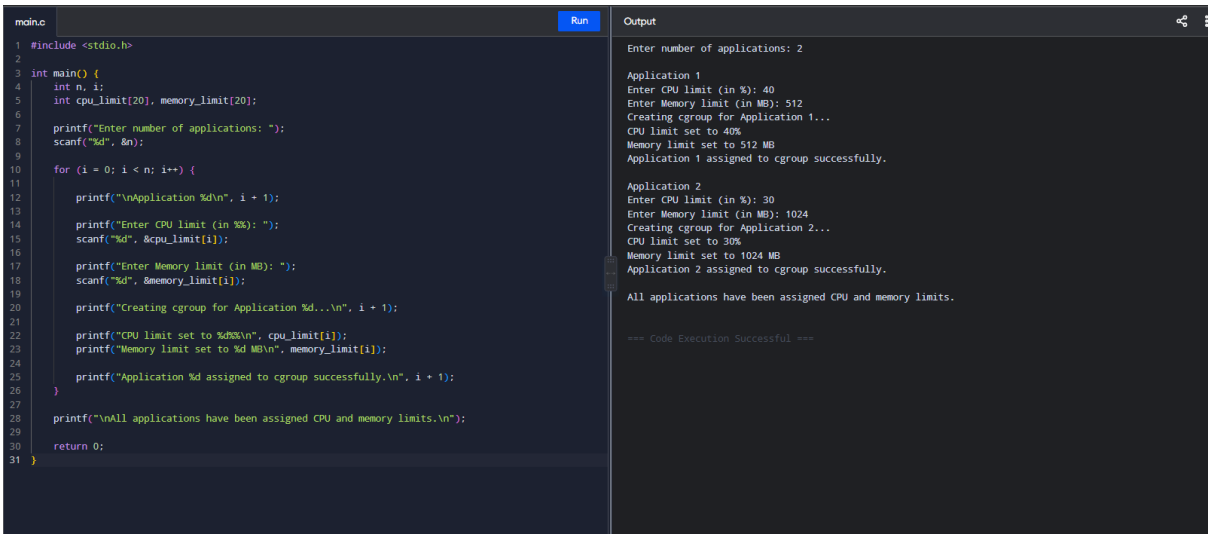
ENDFOR

PRINT "CPU and memory limits applied successfully"

END

Working

1. The algorithm takes a list of applications as input.
2. For each application, a separate group is created.
3. A maximum CPU usage limit is assigned.
4. A maximum memory usage limit is assigned.
5. The application's process is assigned to its respective group.
6. This ensures that no single application consumes excessive CPU or memory resources.



Result: The system maintains balanced resource utilization and improves overall performance by controlling CPU and memory usage of each application.